## 🚀 Proposed Architecture: "NeuroGuard-X"

A modular, bias-resistant, multi-stage Trojan detection and reasoning pipeline.

### 🧩 Stage 1 — Graph Construction & Feature Engineering

**Goal:** Convert RTL/netlist into rich graph representations.

- **Input:** RTL/netlist from TrustHub, HT-Pred, or custom designs.
- **Process:**
    - Parse using **PyVerilog + Yosys** → generate **multi-view graphs**:
        - **Structural graph:** gates + connections
        - **Functional graph:** signal flow + control dependencies
        - **Temporal graph:** activation patterns from simulation traces
    - Extract node features:
        - Gate type, degree, activation probability, distance to outputs
        - Logic entropy, fan-in/fan-out, toggle rate
        - Contextual embeddings (from code tokens via CodeBERT or LLM encoder)

### 🧠 Stage 2 — Multi-View GNN Ensemble

**Goal:** Detect anomalies from multiple perspectives.

- Combine **GCN + GraphSAGE + GAT** in an ensemble:
    - GCN → captures global topology
    - GraphSAGE → handles unseen designs (inductive learning)
    - GAT → learns neighbor importance (attention)
- Fuse outputs via **weighted voting or transformer-based fusion layer**.
- Output: Suspicious node probabilities + graph-level Trojan likelihood.

### 🧱 Stage 3 — Context Expansion & Semantic Encoding

**Goal:** Prepare interpretable context for LLM reasoning.

- Expand around top-k suspicious nodes (2-hop or adaptive radius).
- Encode structural context as **JSON or tabular summaries**.
- Add **semantic metadata**:
    - Gate roles (trigger/payload/control)
    - Signal paths to critical outputs
    - Activation rarity metrics

### 🗣️ Stage 4 — LLM-Driven Structured Reasoning

**Goal:** Explain, validate, and suggest fixes.

- Feed context into an LLM (e.g., GPT-4, Claude, or local model).
- Use **structured prompts**:
    - “Analyze the following hardware region. Identify potential Trojan behavior, explain reasoning, and propose mitigation.”
- LLM outputs:
    - **Verdict:** Trojan / benign / uncertain
    - **Explanation:** human-readable reasoning
    - **Fix suggestion:** code-level or design-level remediation

### 🔁 Stage 5 — Consistency & Bias Control

**Goal:** Ensure reliability of LLM verdicts.

- Perform **multi-orientation querying** (like your friend’s swap-consistency) but enhanced:
    - Add **cross-model consistency** (query two LLMs or two temperature settings).
    - Use **confidence scoring** based on agreement metrics.

- Verdict categories:
    - **Consistent & Correct** → accepted
    - **Inconsistent but plausible** → flagged for review
    - **Inconsistent & incorrect** → discarded

### 📊 Stage 6 — Feedback Learning Loop

**Goal:** Continually improve both GNN and LLM.

- Feed verified LLM explanations back into GNN training as **soft labels**.
- Use **contrastive learning** to align GNN embeddings with LLM semantic embeddings.
- Over time, the system learns to detect Trojans with less LLM intervention.

| Dimension | NeuroGuard-X | Friend's Architecture |
|---|---|---|
| **Bias Control** | Multi-orientation + cross-model consistency | Slot-swap only |
| **Explainability** | Structured reasoning + semantic embeddings | Prompt-based reasoning |
| **Scalability** | Multi-view GNN ensemble handles unseen designs | Single GNN ranker |
| **Adaptivity** | Feedback loop retrains models | Static pipeline |
| **Feature Depth** | Structural + functional + temporal features | Structural only |
| **Integration** | Tight coupling between GNN and LLM embeddings | Sequential hand-off |